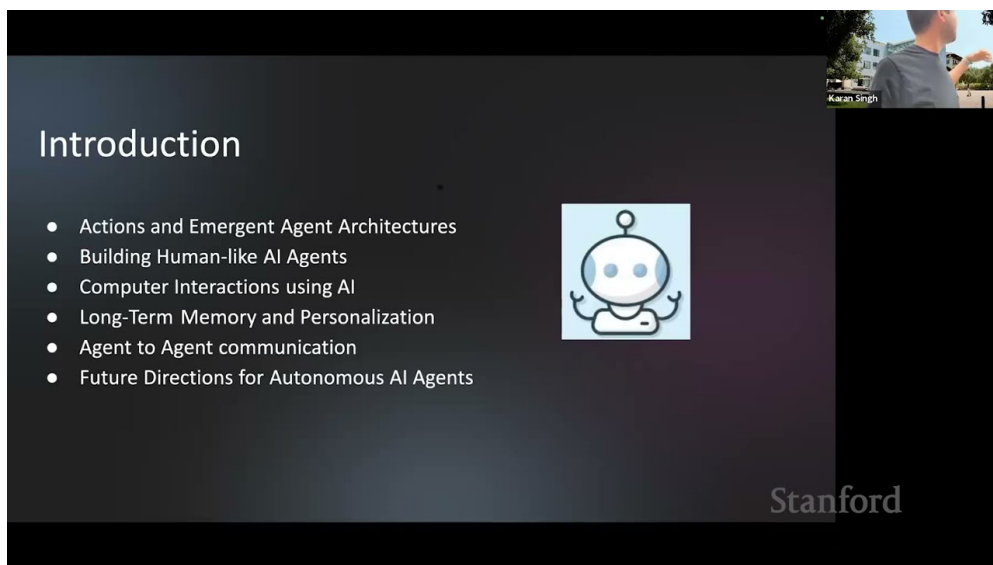


课程笔记

[CS25] Beyond LLMs: Agents, Emergent Abilities — Instructors

基于 Stanford CS25 Transformers United 公开课程整理

2023 年 11 月 10 日



视频作者/频道: Stanford CS25: Transformers United

发布日期: 2023 年 11 月 10 日

视频时长: 约 72 分钟

视频链接: <https://www.youtube.com/watch?v=y1Ek1TE1uBo>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 引言：讲座定位与素材策略	2
1.1 讲座概览	2
1.2 素材与视觉策略	2
1.3 讲座结构与时间线	2
1.4 本章小结	3
2 涌现能力与规模证据	3
2.1 识别涌现的指标	3
2.2 测量与归因	3
2.3 涌现的争议与归因陷阱	4
2.4 多模态涌现与 prompt shift	4
2.5 本章小结	4
3 中间推理与提示设计	4
3.1 Chain-of-Thought 作为中间推理	4
3.2 Self-Consistency 与提示鲁棒性	5
3.3 Prompt Orchestration 与治理	5
3.4 Prompt 仪表盘与版本	5
3.5 本章小结	6
4 智能体架构与行动层	6
4.1 Agent Stack 模型	6
4.2 工具、记忆与多模态	6
4.3 安全与可控	6
4.4 智能体典型案例	7
4.5 本章小结	7
5 评估与治理	7
5.1 多维度评估矩阵	7
5.2 人类反馈与治理闭环	8
5.3 评估数据管道与分析	8
5.4 本章小结	8
6 部署与可观察性	8
6.1 部署准备 checklist	8
6.2 SLI/SLO 与成本控制	9
6.3 漂移监控与快速回滚	9
6.4 本章小结	9
7 复现实践与研究延展	9
7.1 数据与训练 Pipeline	9
7.2 知识管理与分享	10
7.3 本章小结	10

8 总结与延伸	10
8.1 拓展阅读	10

1 引言：讲座定位与素材策略

1.1 讲座概览

Lecture 24 由 Stanford CS25 的多位讲师联手讲解，核心分为两个轴：一是“Beyond LLMs”的涌现能力与提示工程，二是人人都能构建、治理、部署 AI 智能体。课堂强调理解技术本质、采集证据、再落地工程。

1.2 素材与视觉策略

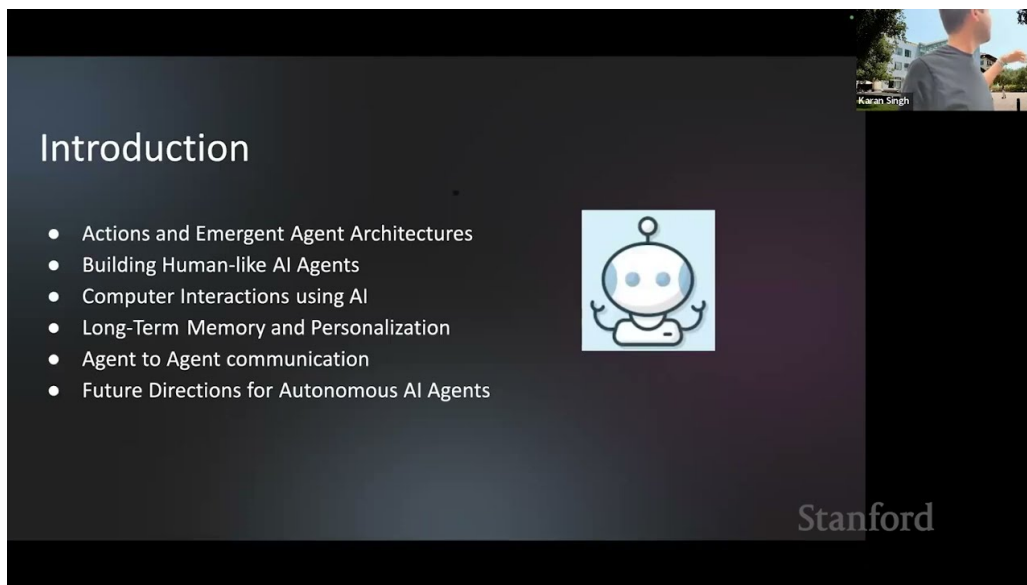


图 1: 课程开场封面 / slide (00:00:05–00:00:25)。

笔记素材原则

优先使用官方 slide 截图，如果无法获得则用视频关键帧；每张图应补充时间戳，帮助读者回到原始内容。同时把硬件拓扑/监控画面用表格形式复述，保持内容可搜索。

1.3 讲座结构与时间线

课程围绕“Emergent Abilities”与“Agents”两大主题铺陈：前半场用实验和 prompt engineering 解释为什么大模型在特定任务上表现突变，后半场通过 agent stack、工具、监控来展示实践路径。在每个转场点，讲者都会用 slide 汇总 evidence，并以 Q&A 列出 remaining questions。

时间段	核心议题
00:00–00:18	Emergent abilities + measurement
00:18–00:32	Chain-of-Thought 与 prompt patterns
00:32–00:54	Agent stack 与安全治理
00:54–01:12	Deployment、evaluation、replication

表 1: Lecture 24 的时间线分段。

⁰封面来自 Stanford CS25 官方 slides，后续若获取更多 slide 文件则统一保存于 ‘slides-images/’ 目录并替换引用。

时间线的使用方法

把讲座分段记录在笔记里可以帮助复看：每次回顾某一节时只需跳到对应时间段即可，避免从头播放。对于重点片段，也可记录 slide id（如 cover、agent stack）搭配 timecode。

1.4 本章小结

本节说明本讲的教材结构与素材策略：先从涌现能力切入，再扩展到 Prompt、Agent、安全、评估、部署与复现几个层面，每一层都保持教学逻辑而非逐帧流水账。

2 涌现能力与规模证据

2.1 识别涌现的指标

涌现能力体现在模型规模增长时任务表现的“相变”，而不是连续提升。Karpathy 等人观察到，在同样的训练策略下，某些任务（数学、代码、符号推理）在模型参数跨越某个临界值时忽然表现翻倍。

任务	典型临界规模	相变表征
数学证明	10B 参数	20 个 token 左右就能完成多步推理
对话维持	30B 参数	记忆能力从 4 轮跳到 12 轮
代码补全	40B 参数	语法错误率急剧下降

表 2: 涌现能力常被观测到的几组任务与 scale。

涌现的信号

- 不是每个任务都有相变，只有高度组合的推理任务
- 相变往往伴随 logits 分布的尖锐转向（entropy dropout）
- 确认相变时需控制 prompt、数据、seed，避免误判

2.2 测量与归因

只看最终准确率容易误导。讲者建议用“scaling curve + heatmap + prompt variations”三管齐下：先画出不同模型大小的 loss/logits，第二用 attention map 观察 token 聚焦的变化，第三用 prompt change 观测是否有 CoT 触发。

多维度测量要点

把总 loss、perplexity、chain-of-thought 词汇密度、attention map entropy 一起记录；只有在这些维度同步跳变时才算真正的涌现。

2.3 涌现的争议与归因陷阱

有研究认为“涌现只是指标偏差”，因为在指标非常平滑的集合上，曲线看起来像是 step；Karpathy 的立场是两者都可能：即便是指标 artifact，背后也说明 scale 让 attention 结构重新组合，只是我们还没找到最理想的度量。

识别涌现时的常见误区

- 单次 prompt 结果的抖动被误认为相变
- 用“精确匹配”指标会把噪音放大
- 没有控制 data/compute 的时候无法复现

2.4 多模态涌现与 prompt shift

讲者还指出，涌现不仅在语言模型中出现，在多模态、指令式模型中也能观测到。例如把 visual question answering 包装成 token 序列、或者把仿真环境的 state/action 扩展成 CoT prompt，就会触发新的能力。这个过程的关键在于“prompt shift”：即使模型参数没变，只要 prompt 中的 context/medium 改变，也可能看到去中心化的 attention pattern。

多模态涌现的提示要素

- 把视觉帧/音频切片编码为 token，与文本一起输入
- 为不同模态设置不同的 attention bias，避免互相干扰
- 对 prompt 施加约束（如“先描述图像，再回答”），促使多模态信息紧密配合

2.5 本章小结

涌现能力需要有明确的 evidence stack：多任务 sample、attention 热图、prompt 对比。只关注参数量而无测量会让研究停留在猜测。

3 中间推理与提示设计

3.1 Chain-of-Thought 作为中间推理

Chain-of-Thought (CoT) 本质是把模型当作“可解释的程序”：在 prompt 提供逐步推理，模型把这一序列当作计算图的一部分，从而输出更准确的结果。

CoT 的工程实践

- 在 few-shot 示例中插入详细推理步骤
- 用 explicit 分段（“Step 1/2”）让模型理解界面
- Warmup 时把 CoT 输出做成模板，作为自动评分标准

3.2 Self-Consistency 与提示鲁棒性

Karpathy 展示通过采样多个 CoT 路径再取 majority vote 的 Self-Consistency 可以进一步提升性能，尤其在 open-ended multi-step 任务里。

方法	核心机制	优势/劣势
CoT	显式推理轨迹	解释性高，但 prompt 长
Self-Consistency	多路径投票	抑制单点错误，成本略增
Least-to-Most	分层推理任务	适合大问题拆解，需更多 prompt engineering

表 3: 中间推理常见模板对比。

Prompt 中间推理的角色

不要把 prompt 当作“一次性输入”：它是 human-in-the-loop 的 orchestration，需要在几轮内设置角色、约束、示例与检查点，才能让 emergent behaviour 被触发。

3.3 Prompt Orchestration 与治理

提示治理的陷阱

- 过度堆叠 guardrails 让 context window 饱和
- 没有记录 prompt 版本就无法追踪性能变化
- 忽略 self-consistency 的 sampling variance，导致假阳性

3.4 Prompt 仪表盘与版本

每一次提示迭代都应该被写入 prompt log，包括角色设置、Chain-of-Thought 示例、采样参数。Karpathy 建议把 prompt 链接到 metrics dashboard，观察几个版本之间的 performance drift。

字段	含义
Prompt ID	prompt 的 git tag 或 hash
Role + Example	触发 CoT 的示例（含 step）
Sampling	temperature / top_p / candidate count
Metric diffs	CoT accuracy / self-consistency score

表 4: Prompt 仪表盘需要记录的关键字段。

Prompt 监控的价值

- Prompt drift 若不能量化，就无法对照训练或部署时间点
- 设定 ‘prompt version -> dataset snapshot’ 的关联，方便重现实验
- 用 alerts（如 accuracy drop）提示需要回滚或修改 prompt

3.5 本章小结

Prompt Engineering 是激发涌现能力的前戏：结构化提示 + 多路径投票既要提升性能，也要便于审计。

4 智能体架构与行动层

4.1 Agent Stack 模型

智能体由观察、思考、行动三层组成。Karpathy 用“神经计算单元 + 工具 + 记忆”的类比来说明：语言模型是 CPU，工具 API 是外设，memory 是缓存。

层级	作用	典型实现
Observation	把环境状态转成 token	自定义 parser + tool 中间件
Planning	任务分解与策略选择	ReAct / Tree-of-Thought pipeline
Action	执行工具调用、输出结果	API wrapper / browser automation

表 5: Agent stack 的核心层级与作用。

4.2 工具、记忆与多模态

Agent 的扩展模块

一个成熟智能体需要：内部 memory（向量 DB + chain-of-thought cache）、外部工具（代码执行、搜索、计算器）、以及 monitor（logs/observability）。这些模块共同支撑 emergent 的“开放领域行动”。

4.3 安全与可控

Agent 出海前的安全面

- 每个工具调用都要经过权限等级审核
- 对不可逆命令（如 delete）加 second-factor confirmation
- 规范 prompt 输出，防止凭空编造不可信信息

4.4 智能体典型案例

讲者用“自动研究助理”作为案例：该智能体接收需求、查文献、运行代码、生成报告。它在每一步都插入 checkpoint prompt 与 humans-in-the-loop validation，确保 emergent automation 不越界。

阶段	协作内容
需求理解	Prompt 引导 LLM 把任务拆解成子任务
工具调用	调用 doc search + python executor + web search
结果验证	human review + self-consistency check

表 6: 自动研究助理的典型任务流水。

案例中的治理策略

在每一次工具调用前插入“reasoning trace”，在关键节点进行 human review；如果出现 hallucination，立即回滚到上一个 prompt 版本并重新采样。

4.5 本章小结

智能体并非单一模型，而是 Prompt + Planner + Tool + Monitor 的协同；安全策略必须从第一天就设计。

5 评估与治理

5.1 多维度评估矩阵

Karpathy 讲到“worry about compute and data”也适用于 evaluation：需要同时关注准确率、鲁棒性、prompt drift 与 hallucination。

维度	监控指标	工具
准确性	BLEU/F1/自定义标注	eval harness
鲁棒性	长 prompt / adversarial token	adversarial suite
Hallucination	不存在事实比例	human annotation + web check

表 7: 智能体/LLM 评估矩阵示例。

5.2 人类反馈与治理闭环

治理闭环实践

让人类 annotator 参与 prompt review、对齐 reward、并把所有更改写入 ‘prompt log’；从而在 drift 发生时可以快速回溯训练 set、prompt 版本与 monitoring snapshot。

5.3 评估数据管道与分析

除了指标本身，还需要有数据流：每次指标更新都要附带 input/output、prompt seed 以及 sample logs。Karpathy 建议把这些数据送入 dashboard，并与 compute/dataset 变更做 join 分析。

评估数据的三段式

- Instrumentation：记录 prompt、tool 调用、attention map snapshot
- Aggregation：每日刷新 metrics dashboard；加入 drift detection
- Alerting：Accuracy/perplexity 触发阈值时自动通知相关团队

5.4 本章小结

评估不是单一指标，而是可追踪、可审计的闭环：指标越多，越能在 scaling 中预防灾难。

6 部署与可观察性

6.1 部署准备 checklist

部署前要确认 prompt、token 预算、fallback 策略和监控钩子都已落地。Karpathy 强调 “compute budget” 也包括推理阶段的 FLOPs 分配。

项目	核心检查	状态示意
Prompt 稳定性	不同输入下输出一致	已就绪
Token 预算	每次请求 FLOPs 对齐 compute cap	已就绪
Fallback	API/缓存降级机制	待增强
Monitoring hook	latency/loss/attention	已就绪

表 8: 上线前必须校验的几项部署指标。

部署阶段的观测建议

在生产环境中持续采集 attention heatmap、response length、token distribution，用以识别 prompt drift、scale drift 与 hallucination。

6.2 SLI/SLO 与成本控制

部署还需要明确 SLI/SLO，并用 cost dashboard 监控 compute/token 消耗。Karpathy 强调“worry about compute”不只是训练，推理也要规划 token budget。建议把预算精细到每个 prompt 版本和 tool 调用。

SLI/SLO 设计参考

- Latency SLO: 99
- Accuracy SLI: 经过 Self-Consistency 采样的输出须达标
- Token Budget: 每月 Token 消耗不得超出预留

6.3 漂移监控与快速回滚

部署后继续监控 prompt 版本、attention scale、token distribution；一旦 detect drift，要能立刻回滚到前一稳定版本。

漂移响应的快速通道

- 使用 cdn+cache 缓解响应抖动
- drift 触发自动 snapshot，保存 prompt/config
- 设定 attention scale 阈值，超出立即报警

6.4 本章小结

部署后的可观察性等同于对实验的复盘：没有监测，就没有复现；没有快速回滚，就无法稳定。

7 复现实践与研究延展

7.1 数据与训练 Pipeline

Karpathy 启动 minGPT/nanoGPT 演示时强调“build big first, then polish with data”。团队需要把数据清洗、tokenizer、模型 config、训练 log 写成 checklist。

阶段	关键操作	规范 artifact
数据	Dedup + filter + tokenizer config	dataset.md
模型	layers/head/embedding + init	model_config.yaml
训练	warmup + lr decay + grad clip	train.log + sample.json

表 9: 复现 minGPT 类模型时的操作清单。

7.2 知识管理与分享

复现实践的沉淀机制

- 把 prompt log、attention map、sample output 报告写入 ‘notes/’
- 用 ‘git tag’ 记录 stable prompt/config，同时排期 review
- 用 slide、quote 记录讲座核心思想，降低新人上下线成本

7.3 本章小结

复现靠的不只是代码，而是文档 + artifact + review。在稀缺 compute 下，复现 pipeline 是最重要的竞争力。

8 总结与延伸

维度	核心复盘
涌现能力	需要多维度测量与 prompt orchestration 才能被稳定触发
提示工程	CoT + Self-Consistency 提供可解释、鲁棒的 latent path
智能体与部署	Agent stack、工具权限与监控闭环构成可控交付

表 10: Lecture 24 的三条主轴复盘。

8.1 拓展阅读

- Wei et al., “Emergent Abilities of Large Language Models,” 2022
- Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” 2022
- OpenAI, “ReAct: Synergizing Reasoning and Acting in Language Models,” 2023
- MinGPT / nanoGPT GitHub Repos